SZZ: Identifying Buggy Entities

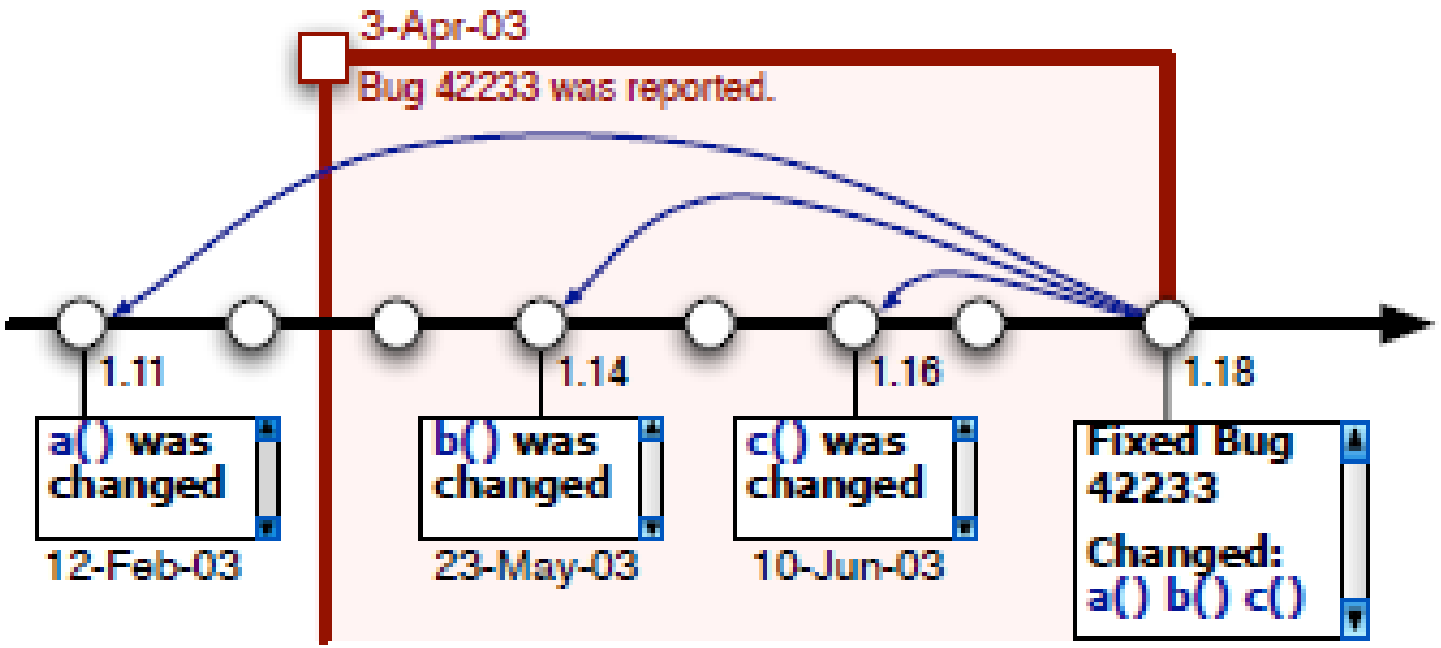
Introduction - context

- Defect prediction aims at identifying software artifacts that are likely to exhibit a defect.
- The main purpose of defect prediction is to reduce the cost of testing and code review, by letting developers focus on specific artifacts.
- Ultimately, the reliability of a prediction model depends on the quality of the dataset.
  - Previous works have identified sources of noise in datasets, including defect misclassification and defect origin, and proposed solutions to deal with them.

INTRODUCTION-SZZ

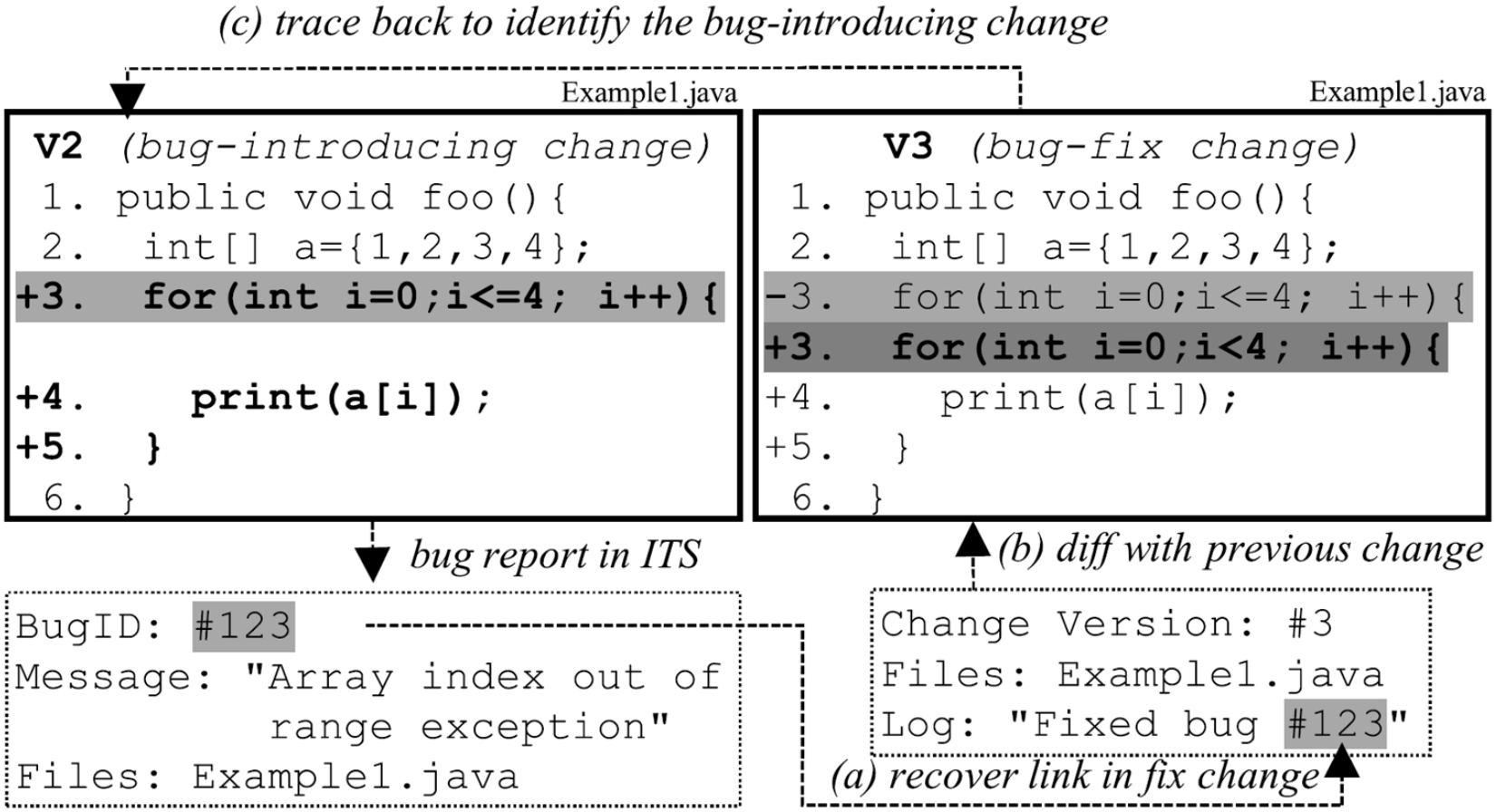
- SZZ : Process to identify when a defect has been actually introduced in a software project
  - Exploits the versioning system annotation mechanism (e.g. git blame)
- Determine, for the source code lines that have been changed in a defect fix, when they have last been changed before such a fix
- Limitations
  - Code additions
  - Merely 26–29% of BICs can be foundusing the algorithm based on the assumption that “a given bugwas introduced by the lines of code that were modified to fix it”.

Example of SZZ



"**When Do Changes Induce Fixes?**"

EXAMPLE OF SZZ



"The Impact of Refactoring Changes on the SZZ Algorithm: An Empirical Study"

SZZ Limitations

- Regressive bugs
- Refactoring
- Inprecise backward search
- Snoring

References

- See papers.